

Module 5 — Splunk: Working with Multivalue Fields

Training Notes for Participants

What is a Multivalue Field?

A **multivalue (MV) field** is a single field that holds **more than one value** in the same event. Splunk creates them automatically in some cases (e.g. multiple email recipients, multiple tags, repeated headers), and you can also create them yourself.

A useful mental model to share with participants:

- A **single-value field** is one box with one item: `recipient = "alice@corp.com"`.
- A **multivalue field** is one box holding a list: `recipients = ["alice@corp.com", "bob@corp.com", "carol@corp.com"]`.

This module covers three things you'll do constantly with MV fields:

1. **Analyze and format** them using multivalue `eval` functions.
2. **Create** them from a single value using `makemv`.
3. **Flatten** them into separate events using `mvexpand`.

Key distinction to emphasize early: multivalue `eval` functions operate **within a single event** (the event count stays the same). `mvexpand` **creates new events** (the event count grows). Choosing the right one is the most common decision point in this module.

1. Multivalue `eval` Functions

These let you count, slice, filter, combine, sort, and reformat the values inside an MV field — all without changing the number of events.

`mvcount(X)` — how many values?

Returns the number of values in `(X)`. Returns `1` for a single value, and NULL if the field has no values.

```
spl
... | eval recipient_count = mvcount(recipients)
```

`mvindex(X, START, END)` — pick value(s) by position

Indexes are **0-based**. `END` is optional and **inclusive**. Negative positions count from the end.

spl

```
... | eval first      = mvindex(recipients, 0)      | first value
... | eval last       = mvindex(recipients, -1)     | last value
... | eval firstTwo   = mvindex(recipients, 0, 1)   | first two values
```

`mvfilter(X)` — keep only matching values

Filters an MV field using a boolean expression. **It can reference only one field**, used directly inside the expression.

spl

```
... | eval gmail_only = mvfilter(match(recipients, "@gmail\.com$"))
... | eval big_nums   = mvfilter(values > 100)
```

`mvfind(MVFIELD, "REGEX")` — find position of a match

Returns the **0-based index** of the first value matching the regex, or NULL if none match.

spl

```
... | eval idx = mvfind(recipients, "@corp\.com$")
```

`mvjoin(MVFIELD, STR)` — multivalue → single string

Joins all values into one string separated by `STR`.

spl

```
... | eval all_recipients = mvjoin(recipients, "; ")
    | -> "alice@corp.com; bob@corp.com; carol@corp.com"
```

`mvappend(X, ...)` — combine values/fields

Builds a single MV field from any mix of literals and fields.

spl

```
... | eval full_list = mvappend("admin@corp.com", recipients, cc_list)
```

`mvdedup(X)` — remove duplicate values

Removes duplicates while preserving order.

```
spl
... | eval unique_tags = mvdedup(tags)
```

`mvsort(X)` — sort the values

Sorts values in ascending lexicographic order.

```
spl
... | eval sorted_tags = mvsort(tags)
```

`mvrange(START, END, STEP)` — generate a numeric MV field

`START` is inclusive, `END` is **exclusive**, `STEP` is optional.

```
spl
... | eval odds = mvrange(1, 10, 2)          | -> 1, 3, 5, 7, 9
```

`mvmap(X, Y)` — transform each value

Applies expression `Y` to every value of `X`.

```
spl
... | eval upper_tags = mvmap(tags, upper(tags))
```

`mvzip(X, Y, "Z")` — pair up two MV fields

Combines two MV fields value-by-value using delimiter `Z` (default comma).

```
spl
... | eval pairs = mvzip(hosts, ports, ":")
    | -> "host1:80", "host2:443"
```

Related: `split(X, "Y")`

The string function `split` is the quickest way to turn a delimited **string** into a multivalue field inside `eval` (an alternative to the `makemv` command in Section 2).

```
spl
```

```
... | eval recipients = split(recipient_string, ",")
```

2. The `makemv` Command

`makemv` converts an existing **single-value** field into a **multivalue** field by splitting it — either on a delimiter or with a tokenizing regex.

Split on a delimiter

```
spl
```

```
... | makemv delim="," recipients  
... | makemv delim=";" tags
```

┆ If you don't specify `delim` or `tokenizer`, the **default delimiter is a single space**.

Split with a regex tokenizer

`tokenizer` takes a regex with a capturing group; each match becomes one value. This is handy when the values aren't separated by a clean delimiter.

```
spl
```

```
... | makemv tokenizer="(\w+)" raw_field
```

Useful options

Option	Purpose
<code>delim="<str>"</code>	Delimiter to split on (default: single space)
<code>tokenizer="<regex>"</code>	Regex with a capture group; each match becomes a value
<code>allowempty=true</code>	Keep empty values produced by the split (default <code>false</code>)
<code>setsv=true</code>	Also keep the original combined value as a single-value version

Example

```
spl
```

```
index=mail
| makemv delim="," recipients
| eval recipient_count = mvcount(recipients)
```

Reverse operation: the `nomv` command collapses a multivalue field back into a single value — useful when you want the field as one string again: `... | nomv recipients`.

3. The `mvexpand` Command

`mvexpand` takes a multivalue field and creates a **separate event for each value**. Every other field in the original event is **copied** onto each new event. This is what you use when you want to run `stats`, `chart`, `top`, etc. **per value**.

Basic syntax

```
spl
... | mvexpand recipients
```

If one event has `recipients = [alice, bob, carol]`, after `mvexpand` you get **three events**, identical except that `recipients` holds `alice`, `bob`, and `carol` respectively.

Limiting the expansion

```
spl
... | mvexpand recipients limit=100
```

Option	Purpose
<code>limit=<int></code>	Max number of values to expand per event (<code>0</code> = no limit)

Things to flag for participants

- `mvexpand` works on **one field at a time** — you cannot expand two MV fields in the same command.
- It can multiply your event count quickly, so it has a memory ceiling (`max_mem_usage_mb`). On very large MV fields, use `limit` or filter first.
- It is the standard bridge between MV data and aggregation: **expand, then `stats`**.

Example — count events per recipient

```
spl

index=mail
| makemv delim="," recipients
| mvexpand recipients
| stats count by recipients
| sort -count
```

Putting It Together — A Realistic Workflow

A single search that uses all three capabilities on an email log where `recipients` arrives as one comma-separated string:

```
spl

index=mail sourcetype=smtp
| makemv delim="," recipients           ``` create the M
| eval recipient_count = mvcount(recipients)           ``` analyze
| eval external_rcpts = mvfilter(NOT match(recipients, "@corp\.com$"))
| eval first_recipient = mvindex(recipients, 0)
| eval recipient_string = mvjoin(recipients, "; ")
| mvexpand recipients           ``` one event p
| stats count AS messages_received by recipients
| sort -messages_received
```

In order, this: **creates** the MV field with `makemv`, **analyzes/formats** it with `mvcount`, `mvfilter`, `mvindex`, and `mvjoin`, then **flattens** it with `mvexpand` so each recipient can be aggregated independently.

Choosing the Right Tool — Quick Decision Guide

You want to...	Use
Turn a delimited string into a list	<code>makemv</code> (or <code>split</code> in <code>eval</code>)
Count / slice / filter values without changing event count	Multivalue <code>eval</code> functions
Display all values as one readable string	<code>mvjoin</code>

You want to...	Use
Run <code>stats</code> / <code>chart</code> / <code>top</code> per individual value	<code>mvexpand</code> , then aggregate
Collapse a list back into one string	<code>nomv</code>

Practice Exercises for Participants

1. **makemv:** Convert a pipe-delimited `error_codes` field into a multivalue field.
2. **mvcount:** Add a field showing how many tags each event has, then find events with more than 5 tags.
3. **mvindex:** Extract the first and last value from a multivalue `path_segments` field.
4. **mvfilter:** From a multivalue `ports` field, keep only ports below 1024 (well-known ports).
5. **mvjoin:** Produce a single readable, semicolon-separated string of all values in `recipients`.
6. **mvexpand:** Expand a multivalue `category` field and produce a `stats count by category`.
7. **Combine:** Starting from a comma-separated `users` string, find the top 10 most frequent users across all events.

Quick Reference Cheat Sheet

Group	Key Functions / Commands
Multivalue functions <code>eval</code>	<code>mvcount</code> , <code>mvindex</code> , <code>mvfilter</code> , <code>mvfind</code> , <code>mvjoin</code> , <code>mvappend</code> , <code>mvdedup</code> , <code>mvsort</code> , <code>mvrangle</code> , <code>mvmap</code> , <code>mvzip</code> , <code>split</code>
Create an MV field	<code>makemv</code> (with <code>delim</code> / <code>tokenizer</code>), <code>split</code>
Flatten an MV field	<code>mvexpand</code> (one event per value)
Collapse back to single value	<code>nomv</code>